IT STUDY HUB

COMPLETE R PROGRAMMING SERIES

MODULE 2: R FUNDAMENTALS & DATA TYPES

Authors:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

WELCOME TO MODULE 2

About This Module

Welcome to Module 2 of the **IT Study Hub Complete R Programming Series**. In the previous module, we configured our environment and learned the basic mechanics of RStudio. Now, we dive into the fundamental building blocks of the language itself: variables, data types, and operators.

Understanding data types is the single most critical skill for a beginning data analyst. If you do not understand how R categorizes information—whether as a numeric value, a character string, or a logical boolean—your code will behave unpredictably. This handbook breaks down R's atomic architecture using simple analogies, clear syntax diagrams, and a comprehensive hands-on project to cement your learning.

IT STUDY HUB

Learning Outcomes

By the completion of this comprehensive module, you will be able to:

- Create, modify, and effectively name variables using proper assignment operators.
- Distinguish between and manipulate numeric, character, logical, and factor data.
- Identify and handle missing or impossible data states (NA, NULL, NaN, Inf).
- Inspect object structures using `class()`, `typeof()`, and `is.*()` functions.
- Force data from one type into another via explicit type coercion (conversion).
- Perform complex mathematical calculations using arithmetic operators.
- Construct logical decision-making tests using relational and logical operators.

Module Coverage

This handbook thoroughly covers the following core topics:

1. Variables and assignment
2. Constants
3. Numbers and arithmetic operations
4. Character/string data
5. Logical/Boolean values
6. Factors
7. NA, NULL, NaN, Inf
8. class() vs typeof()
9. is.*() functions
10. Type conversion (coercion)
11. Arithmetic operators
12. Relational operators
13. Logical operators
14. Assignment operators

IT STUDY HUB

MAIN NOTES

1. Variables and Assignment

CONCEPT:
Variables are named containers used to store data in the computer's memory for later use and manipulation.

Deep Dive Explanation:
In R, a variable allows you to save a value (like a number, text, or a whole dataset) under a specific name. Whenever you need that value, you call the variable by its name instead of typing the raw data again. R is dynamically typed, meaning you don't have to declare a variable's type before assigning a value to it; R figures it out automatically based on the data you provide.

Real-Life Analogy:
Think of a variable as a labeled moving box. The data is the item you put inside the box, and the variable name is the label you write on the outside with a marker. When you need the item, you just look for the label.

Syntax Structure:

```
variable_name <- value
```

R Code Example

```
player_name <- "Alex"
score <- 1500

# Using the variables
print(player_name)
print(score * 2)
```

Console Output

```
[1] "Alex"
[1] 3000
```

Explanation of Code:

We created a variable `player_name` and stored text in it. We created `score` and stored a number. We then retrieved `score`, multiplied it by 2, and printed it without altering the original 1500 stored in memory.

Real-World Use Case:

In a data pipeline, a variable named `raw_data` might hold a 5-gigabyte spreadsheet, allowing you to pass the entire dataset into cleaning functions simply by typing `raw_data`.

Common Mistakes:

Starting a variable name with a number (e.g., `1st_player <- "John"`) or using spaces in the name (e.g., `my score <- 10`). Both will cause syntax errors.

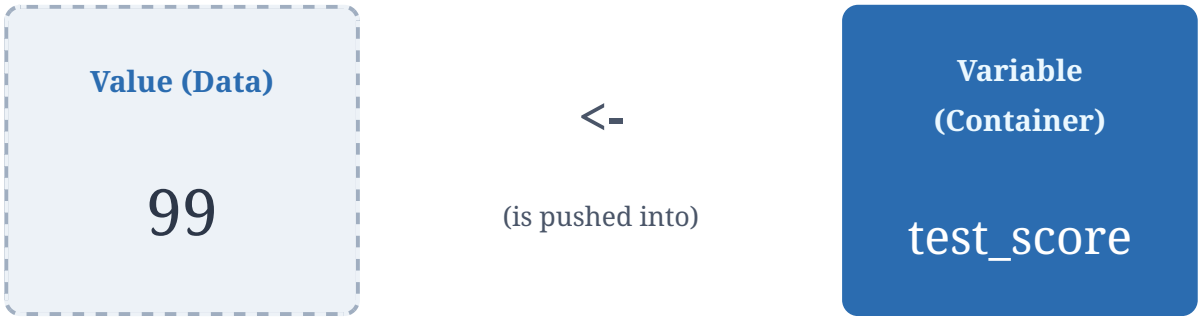
Best Practices:

Use 'snake_case' (e.g., `total_revenue`) or 'camelCase' (e.g., `totalRevenue`) for variable names. Make names descriptive so others understand what the box contains.

Practice Challenge:

Create three variables: `my_age`, `my_name`, and `is_student`. Print them out.

Mental Model: Variables and Assignment



2. Constants in R

CONCEPT:
Constants are values that are built into R and cannot be easily changed or overwritten.

Deep Dive Explanation:
Unlike some languages (like Java or C++) that allow you to define strict custom constants (variables that can never be modified), R relies primarily on built-in constants for universal truths. These include the letters of the alphabet, month names, and mathematical constants like pi. While you can technically overwrite them in your local environment, it is highly discouraged.

Real-Life Analogy:
Constants are like the structural pillars of a house. While variables are the furniture you can move around, constants are built into the architecture and are universally recognized by anyone entering the house.

Syntax Structure:

```
pi
LETTERS
month.name
```

R Code Example

```r
# Printing built-in constants
print(pi)
print(LETTERS[1:5])
print(month.name[1:3])
```

Console Output

```
[1] 3.141593
[1] "A" "B" "C" "D" "E"
[1] "January"  "February" "March"
```

Explanation of Code:

R instantly knows what `pi`, `LETTERS`, and `month.name` represent without us having to define them. We used `[1:5]` to slice the first 5 letters of the alphabet.

Real-World Use Case:

When building time-series plots, data analysts frequently use `month.abb` (built-in constant for Jan, Feb, Mar) to instantly label the X-axis of their charts without manually typing all 12 months.

> **Common Mistakes:**
> Accidentally naming your own variable `pi` or `c`. Since `c` is a foundational function and `pi` is a constant, overwriting them can cause bizarre errors in complex scripts.

Best Practices:
If you need to simulate a custom constant in your own code (like a fixed tax rate), name it using all uppercase letters (e.g., `TAX_RATE <- 0.18`) to signal to other programmers that it shouldn't be altered.

Practice Challenge:
Print the built-in constant `letters` (lowercase) and extract the last 3 letters.

3. Numbers and Numeric Data

CONCEPT:
R handles numeric data in two primary forms: 'numeric' (decimals/doubles) and 'integer' (whole numbers).

Deep Dive Explanation:
By default, any number you type in R is stored as a 'numeric' type, which in computer science terms is a double-precision floating-point number. This means it has decimal places, even if they aren't shown (e.g., 5 is treated as 5.0). If you want to force R to store a number strictly as an integer (a whole number without decimals, which saves memory), you must append a capital 'L' to the end of it.

Real-Life Analogy:
A 'numeric' is a digital scale that weighs things down to the milligram (150.00 lbs). An 'integer' is a simple clicker counter used to count people entering a room (you can't have 1.5 people).

Syntax Structure:

```
x <- 10.5   # Numeric
y <- 10L    # Integer
```

R Code Example

```
weight <- 75.5
count <- 100L

print(class(weight))
print(class(count))
```

Console Output

```
[1] "numeric"
[1] "integer"
```

Explanation of Code:

The `class()` function tells us what type of data the variable holds. Notice that appending the 'L' to 100 changed its classification from the default 'numeric' to 'integer'.

Real-World Use Case:

When working with billions of rows of database IDs (like customer numbers), storing them as Integers rather than Numerics can save gigabytes of RAM in enterprise environments.

Common Mistakes:

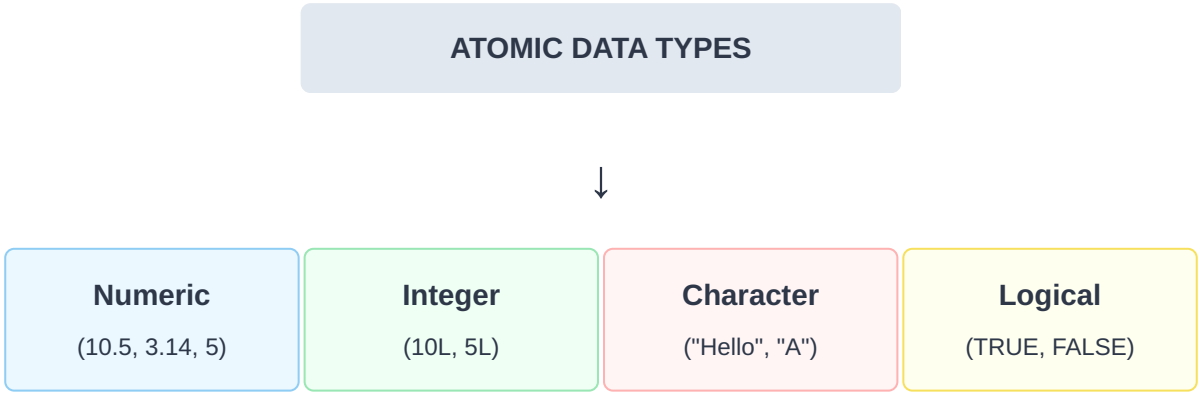
Forgetting that R defaults to numeric. Using integers incorrectly in division can sometimes yield numeric results anyway, confusing beginners.

Best Practices:

Unless memory optimization for massive datasets is a strict requirement, just use standard numeric types for simplicity.

Practice Challenge:
Create an integer variable representing your birth year and check its class.

Hierarchy of R Data Types (Atomic Vectors)



4. Character / String Data

CONCEPT:
Character data represents text. It can be a single letter, a word, or an entire paragraph.

Deep Dive Explanation:
In R, textual data is called a 'character' type (often referred to as 'strings' in other languages). To create a character variable, you must enclose the text in either single quotes (`'`) or double quotes (`"`). R treats both equally, but double quotes are the standard convention. You can use the `paste()` or `paste0()` functions to combine multiple character strings together.

> **Real-Life Analogy:**
> A character string is like a literal string of beads with letters painted on them. The quotes are the knots at the beginning and end that keep the letters from falling off and getting confused with actual R code.

Syntax Structure:

```
my_text <- "Data Science"
my_char <- 'A'
```

R Code Example

```
first_name <- "John"
last_name <- 'Doe'

# Combine strings
full_name <- paste(first_name, last_name)
print(full_name)
```

Console Output

```
[1] "John Doe"
```

Explanation of Code:
R accepts both single and double quotes. The `paste()` function joins the two strings together with a default space between them.

Real-World Use Case:
Character data is used extensively in Sentiment Analysis—analyzing thousands of product reviews (text) to determine if customers are generally happy or angry.

Common Mistakes:
Forgetting the closing quote, causing the R console to show a `+` sign indicating it is infinitely waiting for you to finish the sentence.

Best Practices:
Stick to double quotes (`"`) as a standard. If your text actually contains a quote (e.g., `"He said 'Hello'"`), using double quotes on the outside makes it easy.

Practice Challenge:
Create a variable holding your favorite quote as a character string.

5. Logical / Boolean Values

CONCEPT:
Logical data only has two possible states: TRUE or FALSE. They are the core of decision-making in code.

Deep Dive Explanation:
Logical types (Booleans) represent binary truth values. They are not text, so they are never written in quotes. They are strictly typed as `TRUE` or `FALSE` (all caps). You can also use the abbreviations `T` and `F`. Logicals are usually generated dynamically when you ask R a question, such as 'Is 10 greater than 5?'.

Real-Life Analogy:
A logical value is a basic light switch. It can only be ON (TRUE) or OFF (FALSE). There is no 'maybe'.

Syntax Structure:

```
x <- TRUE
y <- FALSE
```

R Code Example

```
is_raining <- TRUE
check_math <- (10 > 5)

print(is_raining)
print(check_math)
print(class(check_math))
```

Console Output

```
[1] TRUE
[1] TRUE
[1] "logical"
```

Explanation of Code:

We explicitly assigned `TRUE` to `is_raining`. For `check_math`, R evaluated the expression `10 > 5`, realized it was true, and automatically stored it as a logical `TRUE`.

Real-World Use Case:

Logicals are used to filter data. For example, setting `is_premium_subscriber == TRUE` to allow users to bypass a paywall on a news website.

Common Mistakes:

Typing `True` or `true` instead of `TRUE`. R is case-sensitive, and lowercase 'true' will result in an 'object not found' error. Also, accidentally putting quotes around it (`"TRUE"`), which turns it into text.

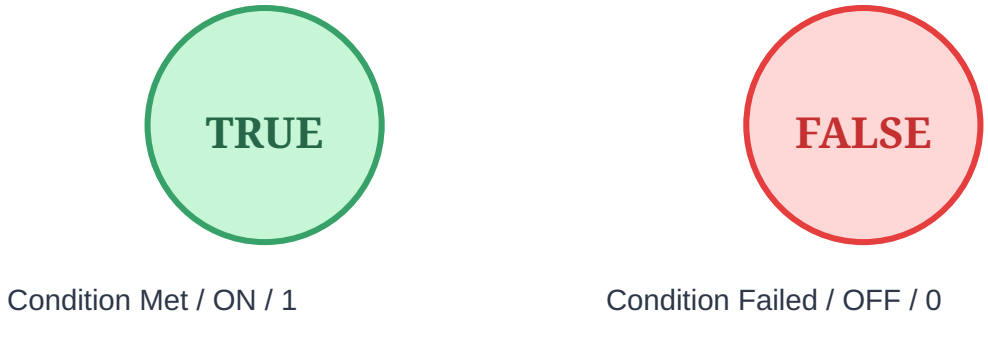
Best Practices:

Always type out `TRUE` and `FALSE` fully instead of using `T` or `F`. While `T` works, it is technically a variable that could be overwritten by bad code, whereas `TRUE` is an unalterable reserved keyword.

Practice Challenge:

Ask R if 50 is less than 20. Store the result in a variable and print its class.

Logical Values Concept



6. Factors (Categorical Data)

CONCEPT:

Factors are a special data type used to store categorical data with predefined, limited values (levels).

Deep Dive Explanation:

While 'character' data is used for open-ended text (like names or addresses), 'factors' are used for data that belongs in specific buckets or categories. Under the hood, R actually stores factors as integers (1, 2, 3...) mapped to character labels. This makes statistical

modeling and plotting much faster and more accurate because R knows exactly how many categories exist in the variable.

> **Real-Life Analogy:**
> A character string is like a blank name tag where you can write anything. A factor is like a multiple-choice survey question where you can only select 'Small', 'Medium', or 'Large'.

Syntax Structure:

```
factor(vector, levels = c(...))
```

R Code Example

```
# A vector of raw text
shirt_sizes <- c("Medium", "Small", "Large", "Small", "Medium")

# Converting to a factor
size_factor <- factor(shirt_sizes, levels = c("Small", "Medium",
"Large"))

print(size_factor)
print(table(size_factor))
```

Console Output

```
[1] Medium Small  Large  Small  Medium
Levels: Small Medium Large
size_factor
 Small Medium  Large
     2      2      1
```

Explanation of Code:
By converting the strings to a factor, R now tracks the unique categories ('Levels'). The `table()` function instantly counts how many times each category appears.

Real-World Use Case:

In clinical trials, patient severity is stored as an ordered factor ('Mild', 'Moderate', 'Severe'). This ensures algorithms treat 'Moderate' as mathematically in between 'Mild' and 'Severe'.

Common Mistakes:

Trying to add a new category to a factor that wasn't in the original 'levels' definition. R will reject it and generate an NA.

Best Practices:

Always use factors for independent variables in machine learning models (like gender, country, or status). Use standard character strings for unique identifiers.

Practice Challenge:

Create a factor of days of the week you attended the gym: 'Mon', 'Wed', 'Fri', 'Mon'.

IT STUD

Comparison: Character vs Factor

Feature	Character String	Factor
Purpose	Free-form text (Names, Addresses)	Categorical groups (Male/Female, High/Low)
Memory Storage	Stored exactly as written text.	Stored as integers (1,2,3) mapped to labels.
Categories (Levels)	No concept of distinct categories.	Strictly enforces pre-defined categories.
Statistical Modeling	Ignored or creates errors.	Essential. R automatically handles dummy variables.

7. The Special Values: NA, NULL, NaN, Inf

CONCEPT:
R has specific keywords to represent missing, undefined, or impossible mathematical states.

Deep Dive Explanation:
Data is rarely perfect. `NA` (Not Available) represents a missing value; the space exists, but it is empty. `NULL` represents an object that simply does not exist. `NaN` (Not a Number) occurs when a mathematical operation is impossible (like 0 divided by 0). `Inf` (Infinity) occurs when a number exceeds computational limits (like dividing by 0).

> **Real-Life Analogy:**
> `NA` is an empty chair at a dinner table (the guest is missing). `NULL` means the chair itself was removed. `NaN` is asking what color the number 5 is (a nonsensical question). `Inf` is asking to count all the stars in the universe.

Syntax Structure:

```
NA, NULL, NaN, Inf
```

R Code Example

```
missing_data <- NA
math_error <- 0/0
infinity <- 10/0

print(missing_data)
print(math_error)
print(infinity)
```

Console Output

```
[1] NA
[1] NaN
[1] Inf
```

Explanation of Code:
R correctly interprets 0/0 as Not a Number, and 10/0 as Infinity. NA remains NA.

Real-World Use Case:
If a patient forgets to fill out their age on a form, the data scientist replaces the blank cell with `NA` before running the analysis so the software doesn't crash.

Common Mistakes:

Trying to use `x == NA` to check for missing values. Because NA is 'unknown', asking if something equals unknown yields 'unknown'. You must use `is.na(x)`.

Best Practices:

Always check your data for `NA` values before running statistical functions like `mean()`, or use the argument `na.rm = TRUE`.

Practice Challenge:

Try to calculate the mean of `c(10, 20, NA)`. See what happens, then look up how to ignore the NA.

The Special States: NA vs NULL vs NaN vs Inf

Keyword	Meaning	Real-World Example	How to Check it
NA	Not Available (Missing data)	Patient didn't answer question #5.	`is.na()`
NULL	Undefined (Object does not exist)	A completely empty list.	`is.null()`
NaN	Not a Number (Impossible math)	Dividing 0 by 0.	`is.nan()`
Inf / -Inf	Infinity / Negative Infinity	Dividing 10 by 0.	`is.infinite()`

8. Inspecting Data: class() vs typeof()

CONCEPT:
These functions inspect what kind of data is stored in a variable, but from two different perspectives.

Deep Dive Explanation:
`class()` tells you the object's high-level, object-oriented classification (how R functions will interact with it). `typeof()` tells you how R is allocating the data in the computer's low-level C-based memory. As a beginner, you will almost exclusively use `class()`, but understanding `typeof()` helps explain how R works under the hood.

Real-Life Analogy:
`class()` tells you that a vehicle is a 'Sports Car' (its real-world behavior). `typeof()` tells you it is made of 'Carbon Fiber and Steel' (its fundamental physical makeup).

Syntax Structure:

```
class(variable)
typeof(variable)
```

R Code Example

```
my_number <- 5.5

print(class(my_number))
print(typeof(my_number))
```

Console Output

```
[1] "numeric"
[1] "double"
```

Explanation of Code:

From a high-level data analysis perspective (`class`), 5.5 is a 'numeric' value. From the computer's memory allocation perspective (`typeof`), it is a 'double' precision floating point.

Real-World Use Case:

When a plot fails to render, a data analyst uses `class()` to ensure the data wasn't accidentally loaded as 'character' instead of 'numeric'.

Common Mistakes:

Confusing the two and relying on `typeof` for standard data analysis. For example, the `typeof` a factor is 'integer', which can deeply confuse beginners.

Best Practices:

Use `class()` 95% of the time. Only use `typeof()` when dealing with highly advanced R programming or C++ integrations.

Practice Challenge:

Check the `class()` and `typeof()` of the string `"Hello"`.

Comparison: class() vs typeof()

Data	class() (High-Level Object View)	typeof() (Low-Level Memory View)
`5.5`	"numeric"	"double"
`5L`	"integer"	"integer"
`"Hello"`	"character"	"character"
`factor("Red")`	"factor"	"integer" (Under the hood!)

9. Checking Data Types: is.*() Functions

CONCEPT:
The is.* functions ask R a simple Yes/No question about a variable's data type.

Deep Dive Explanation:
Instead of fetching the class and manually comparing it, R provides a suite of boolean functions that start with `is.`. For example, `is.numeric()`, `is.character()`, `is.logical()`, and `is.na()`. These functions evaluate the variable and immediately return a logical `TRUE` or `FALSE`.

Real-Life Analogy:
Instead of asking 'What is your job title?' (which returns 'Doctor'), you are asking 'Are you a Doctor?' (which returns TRUE).

Syntax Structure:

```
is.numeric(variable)
```

R Code Example

```
age <- 25
name <- "Dithsa"

print(is.numeric(age))
print(is.character(age))
print(is.na(name))
```

Console Output

```
[1] TRUE
[1] FALSE
[1] FALSE
```

Explanation of Code:
R confirms that `age` is numeric, confirms it is NOT a character, and confirms that `name` is NOT a missing NA value.

Real-World Use Case:
In automated scripts, programmers write 'if statements' using these functions: *If the user input `is.numeric()`, then calculate the total, else show an error.*

Common Mistakes:
Writing `variable == NA` to check for missing data. You MUST use `is.na(variable)` to check for NAs.

Best Practices:
Use `is.*()` functions defensively at the beginning of your scripts to ensure the incoming data is exactly what you expect before processing it.

Practice Challenge:
Create a variable `val <- FALSE` and check if it is logical using `is.logical()`.

10. Type Conversion (Coercion)

CONCEPT:
Coercion is the process of changing data from one type to another, using as.*() functions.

Deep Dive Explanation:
Sometimes data is imported incorrectly (e.g., numbers imported as text like `"100"`). You can force R to change the data type explicitly using functions that begin with `as.`. For instance, `as.numeric("100")` converts the text into a usable math number. R also does 'implicit coercion' automatically behind the scenes if you mix data types in a vector, following a strict hierarchy: Logical -> Integer -> Numeric -> Character.

Real-Life Analogy:
Explicit coercion is like melting down an ice cube (`"100"`) to turn it into liquid water (`100`) so you can use it in a recipe.

Syntax Structure:

```
as.numeric(variable)
as.character(variable)
```

R Code Example

```
text_number <- "500"

# It is currently text, so math fails if we tried text_number + 10
# Convert it explicitly:
real_number <- as.numeric(text_number)

print(class(text_number))
print(class(real_number))
print(real_number + 10)
```

Console Output

```
[1] "character"
[1] "numeric"
[1] 510
```

Explanation of Code:

We successfully forced the text string `"500"` into a numeric format, allowing us to perform mathematical addition on it.

Real-World Use Case:

When importing financial data from Excel, currency symbols ($) often cause R to read the column as text. Analysts use coercion to strip the symbols and convert the column back to numeric.

Common Mistakes:

Trying to coerce letters into numbers (e.g., `as.numeric("Apple")`). R doesn't know what number an Apple is, so it will output `NA` and throw a warning.

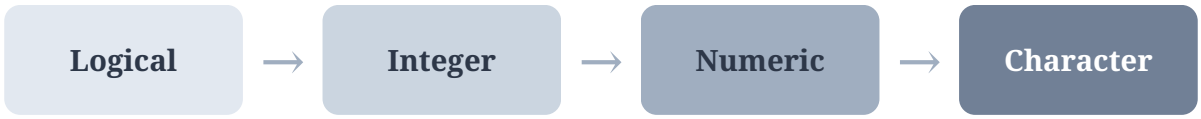
Best Practices:

Always inspect the resulting variable immediately after using an `as.*()` function to ensure you didn't accidentally generate a column full of NAs.

Practice Challenge:
Create a logical variable `x <- TRUE` and convert it to numeric. Observe what number R assigns to TRUE.

Implicit Coercion Hierarchy

If you mix data types, R automatically upgrades them following this path:



Example: Mixing TRUE (Logical) and "Hello" (Character) turns everything into Character.

11. Arithmetic Operators

CONCEPT:
Operators used to perform standard mathematical calculations.

Deep Dive Explanation:
R is fundamentally a statistical engine, meaning it acts as an incredibly powerful calculator. The standard operators are `+` (addition), `-` (subtraction), `*` (multiplication), and `/` (division). Exponents are handled by `^`. Additionally, R has special operators: Modulo `%%` (which returns the remainder of division) and Integer Division `%/%` (which returns division without decimals).

Real-Life Analogy:
These are the standard buttons on a physical calculator, plus a few extra buttons for advanced algebra.

Syntax Structure:

```
a + b
a %% b
```

R Code Example

```
val1 <- 10
val2 <- 3

print(val1 ^ 2)   # Exponent
print(val1 %% val2)  # Modulo (Remainder)
print(val1 %/% val2) # Integer Division
```

Console Output

```
[1] 100
[1] 1
[1] 3
```

Explanation of Code:

10 squared is 100. When 10 is divided by 3, the remainder is 1 (`%%`). When 10 is divided by 3 ignoring decimals, the result is exactly 3 (`%/%`).

Real-World Use Case:

Modulo (`%%`) is heavily used in programming to determine if a number is even or odd (if `x %% 2 == 0`, the number is even).

Common Mistakes:

Forgetting standard order of operations (PEMDAS). `10 + 5 * 2` equals 20, not 30. Use parentheses to enforce order: `(10 + 5) * 2`.

Best Practices:
Use spaces around your arithmetic operators to make the code readable. Write `a + b`, not `a+b`.

Practice Challenge:
Calculate the remainder of 25 divided by 7.

12. Relational Operators

CONCEPT:
Operators used to compare two values, returning a TRUE or FALSE logical result.

Deep Dive Explanation:
Relational operators ask questions about how values relate to each other. They include `>` (greater than), `<` (less than), `>=` (greater than or equal to), and `<=`. Crucially, to test if two things are exactly equal, you must use a double equals sign `==`. To test if they are NOT equal, use `!=`. The result of any relational operation is always a logical `TRUE` or `FALSE`.

Real-Life Analogy:
Relational operators act as a strict bouncer at a club. They look at your ID (Value A), compare it to the age limit (Value B), and output a strict YES (True) or NO (False).

Syntax Structure:

```
a == b
a != b
```

R Code Example

```
x <- 15
y <- 20

print(x > y)
print(x == 15)
print(y != 100)
```

Console Output

```
[1] FALSE
[1] TRUE
[1] TRUE
```

Explanation of Code:

Is 15 greater than 20? False. Is 15 equal to 15? True. Is 20 NOT equal to 100? True.

Real-World Use Case:

Filtering a dataset of millions of customers down to a specific target audience using code like: `filter(data, age >= 18)`.

Common Mistakes:

Using a single equals sign `=` to compare values. A single equals sign assigns a value (like `<-`); a double equals `==` compares them.

Best Practices:

When comparing decimal numbers, be aware of floating-point precision issues. It is often safer to check if the difference between two numbers is very small rather than checking strict `==`.

Practice Challenge:
Assign your age to a variable, then write a relational expression checking if you are `>= 18`.

13. Logical Operators

CONCEPT:
Operators used to combine multiple relational conditions together.

Deep Dive Explanation:
Sometimes you need to ask multiple questions at once. Logical operators combine TRUE/FALSE values. The `&` (AND) operator requires BOTH conditions to be true. The `|` (OR) operator requires AT LEAST ONE condition to be true. The `!` (NOT) operator simply reverses a logical value (TRUE becomes FALSE).

Real-Life Analogy:
AND (`&`) is like buying a house: it must have a pool AND a garage (both required).
OR (`|`) is renting: it must have a pool OR a balcony (either is fine).

Syntax Structure:

```
cond1 & cond2
cond1 | cond2
```

R Code Example

```
age <- 25
has_ticket <- TRUE

# AND: Both must be true
can_enter_club <- (age >= 21) & (has_ticket == TRUE)

# OR: Only one needs to be true
gets_discount <- (age < 18) | (age > 65)

print(can_enter_club)
print(gets_discount)
```

Console Output

```
[1] TRUE
[1] FALSE
```

Explanation of Code:

The person is over 21 AND has a ticket, so they can enter (TRUE). The person is neither under 18 NOR over 65, so they do not get a discount (FALSE).

Real-World Use Case:

Extracting patients from a medical database who are 'Over 50' AND 'Have High Blood Pressure' to invite them to a clinical trial.

Common Mistakes:

Confusing `&` with `&&`. Single `&` is vectorized (works on whole columns of data at once). Double `&&` only looks at the very first element. Stick to single `&` as a beginner.

Best Practices:
Always wrap individual conditions in parentheses before combining them with logical operators. E.g., `(a > b) & (c == d)`. It ensures the code executes exactly as you intend.

Practice Challenge:
Write code checking if `10 > 5 | 100 < 50`.

Operator Categories: Relational vs Logical

Relational Operators (Comparisons)	Logical Operators (Combinations)
Used to compare variables/values.	Used to chain conditions together.
== (Exactly Equal to)	& (AND: both must be TRUE)
!= (Not Equal to)	| (OR: at least one must be TRUE)
>, <, >=, <= (Greater/Less)	! (NOT: reverses TRUE to FALSE)

14. Assignment Operators

CONCEPT:
Operators specifically used to assign data to variables.

Deep Dive Explanation:

The standard assignment operator in R is the left-pointing arrow `<-`. You can technically use the equals sign `=` to assign variables, but it is heavily frowned upon in the R community because `=` is primarily used to assign arguments inside functions. R also allows right-assignment `->` (value -> variable) and global assignment `<<-` (used in advanced function building).

> **Real-Life Analogy:**
> The arrow `<-` visually shows the flow of data: the calculation on the right is being pushed into the box on the left.

Syntax Structure:

```
variable <- value
value -> variable
```

R Code Example

```
a <- 50
b = 60
70 -> c

print(paste(a, b, c))
```

Console Output

```
[1] "50 60 70"
```

Explanation of Code:

All three methods successfully created a variable and assigned a value to it. `a` used the standard arrow, `b` used equals, and `c` used the right-facing arrow.

Real-World Use Case:

Every single R script written by professionals relies on `<-` to cleanly separate variable creation from function arguments.

Common Mistakes:
Using `=` everywhere because you are used to Python or Java. While it works, other R programmers reading your code will find it messy and non-standard.

Best Practices:
Use `<-` exclusively for assigning variables. Use `=` exclusively when passing arguments to functions (e.g., `mean(data, na.rm = TRUE)`). Never use `<<-` until you master functional environments.

Practice Challenge:
Try assigning the number 99 to a variable named `z` using the right-facing arrow `->`.

Comparison of Assignment Methods

Operator	Name	Standard Usage	Verdict
`<-`	Left Assignment	`x <- 10`	**Gold Standard.** Use this 99% of the time.
`=`	Equals Assignment	`x = 10`	**Frowned Upon.** Reserved for function arguments.
`->`	Right Assignment	`10 -> x`	**Rare.** Used occasionally at the end of long pipelines.

HANDS-ON PROJECT

IT STUDY HUB

STUDENT MARKS CALCULATOR

Problem Statement:

You are a university teaching assistant. You need to write an R script that calculates a single student's performance across three exams. The script must define variables, perform mathematical operations, determine the highest score, and use logical operators to figure out if the student passed the course (Average >= 50 AND no single exam score is below 35).

Complete R Code

```
# 1. Variables and Assignment (Numeric & Character)
student_name <- "Dithsa Dutta"
exam1 <- 78
exam2 <- 85
exam3 <- 42   # Oh no, a tough exam!
exam_max <- 100

# 2. Arithmetic Operations (Calculating Totals and Averages)
total_score <- exam1 + exam2 + exam3
average_score <- total_score / 3

# 3. Built-in Math Functions (Highest & Lowest)
highest_mark <- max(exam1, exam2, exam3)
lowest_mark <- min(exam1, exam2, exam3)

# 4. Relational & Logical Operators (Pass / Fail Logic)
# Rule: Average must be >= 50 AND ALL exams must be >= 35
passed_course <- (average_score >= 50) & (lowest_mark >= 35)

# 5. Output Generation (Converting Data Types to Character
implicitly via paste)
print(paste("Student Profile:", student_name))
print(paste("Total Marks:", total_score, "/", (exam_max * 3)))
print(paste("Average Score:", average_score))
print(paste("Highest Score:", highest_mark))
print(paste("Did the student pass? :", passed_course))
```

Expected Console Output

```
[1] "Student Profile: Dithsa Dutta"
[1] "Total Marks: 205 / 300"
[1] "Average Score: 68.3333333333333"
[1] "Highest Score: 85"
[1] "Did the student pass? : TRUE"
```

Step-by-Step Explanation:

First, we used the `<-` operator to assign scores to our variables. Next, we applied standard arithmetic (`+` and `/`) to find the total and average. R has built-in functions like `max()` and `min()` which we used to quickly find the extreme scores. The most crucial part is the pass/fail logic: we used the relational operator `>=` to check conditions, and combined them with the logical & (AND) operator. Because 68.3 is >= 50, AND the lowest score (42) is >= 35, the final `passed_course` variable stored `TRUE`.

Project Extension Challenge:

1. Change `exam3` to `30`. Rerun the script. Does the student fail?
2. Introduce a new variable: `exam4 <- NA` (the student missed the test). Try adding `exam4` to the total. Watch it break. Look up `na.rm = TRUE` for the `sum()` function to fix it!

ASSESSMENTS & INTERVIEW PREP

15+ Practice Questions

Test your fundamental knowledge by answering these questions without looking back.

1. What symbol is the gold standard for assigning a variable in R?
2. If you type `pi` into the console, what happens? Why?
3. What is the default numeric data type in R, and how do you force a number to be an integer?
4. What is the difference between single quotes and double quotes when creating character strings?
5. What are the two possible values for a logical data type?
6. Why are factors preferred over character strings for statistical modeling?
7. What does `NA` stand for in R, and what does it represent?
8. What happens if you divide a number by zero in R? (e.g., 5/0)
9. Explain the difference in output between `class()` and `typeof()`.
10. If `x <- "100"`, what function would you use to change it into a math-ready number?
11. What does the modulo operator (`%%`) do?
12. Evaluate this mentally: `(10 > 5) & (5 == 4)`. True or False?
13. Evaluate this mentally: `(10 > 5) | (5 == 4)`. True or False?
14. Why shouldn't you use `== NA` to check for missing values?
15. What is implicit coercion in R, and what is its hierarchy?
16. How does R evaluate `!TRUE`?

IT STUDY HUB

20+ Viva Questions (With Answers)

These questions are designed for rapid-fire university viva or oral examinations.

Q1: Is R dynamically typed or statically typed?

Answer: R is dynamically typed; you don't need to declare data types before assigning them.

Q2: Can a variable name start with a number in R?

Answer: No, variable names must start with a letter or a dot (not followed by a number).

Q3: What does the 'L' stand for in `10L`?

Answer: It forces R to store the number 10 strictly as an Integer rather than a Numeric (double).

Q4: How do you concatenate two character strings?

Answer: By using the `paste()` or `paste0()` functions.

Q5: Are `TRUE` and `True` the same in R?

Answer: No. R is case-sensitive. `TRUE` is a logical boolean, `True` is an undefined variable.

Q6: What is a factor?

Answer: A data type used to store categorical data with predefined levels.

Q7: What is the difference between NA and NULL?

Answer: NA represents a missing value (empty slot). NULL represents an undefined or non-existent object.

Q8: How do you check if a variable `x` contains NA?

Answer: By using the `is.na(x)` function.

Q9: What does `NaN` mean?

Answer: Not a Number. It occurs during undefined mathematical operations like 0/0.

Q10: What is the result of `class(TRUE)`?

Answer: The string "logical".

IT STUDY HUB

Q11: How do you convert a string to a number?

Answer: Using the `as.numeric()` coercion function.

Q12: What does `%/%` do?

Answer: It performs integer division (division discarding the remainder).

Q13: What does `%%` do?

Answer: It performs modulo division (returns only the remainder).

Q14: Explain `==` vs `=`.

Answer: `==` is a relational operator testing for equality. `=` is an assignment operator.

Q15: What is the `!` operator?

Answer: The Logical NOT operator. It negates a boolean (e.g., `!TRUE` becomes `FALSE`).

Q16: What happens if you add a numeric and a logical in R? (e.g. `TRUE + 5`)

Answer: R implicitly coerces TRUE to 1 (and FALSE to 0). The result is 6.

Q17: Is it best practice to use `<-` or `=` for variables?

Answer: `<-` is the gold standard for variable assignment in R.

Q18: What is implicit coercion?

Answer: When R automatically converts data types to accommodate mixing them (e.g., Logical -> Numeric -> Character).

Q19: Can you assign variables from left to right?

Answer: Yes, using the right-assignment arrow `->`.

Q20: What will `typeof(factor(c("A", "B")))` return?

Answer: It returns "integer", because factors are stored as integers under the hood.

Q21: How do you explicitly test if something is a character?

Answer: By using the `is.character()` function.

10+ Corporate Interview Questions

Prepare for data analyst technical rounds with these conceptual questions.

IT STUDY HUB

1. Explain R's implicit coercion hierarchy. What happens if you combine a character, a numeric, and a logical in a single vector?

2. Walk me through the differences between `NA`, `NULL`, and `NaN`. In what specific scenarios would you encounter each?

3. Why is it dangerous to use `==` when comparing two floating-point numbers in R?

4. When would you strictly prefer storing a variable as an Integer rather than a standard Numeric?

5. Explain the difference between `class()` and `typeof()`. Give an example of when they return different results.

6. How do Factors improve memory efficiency and statistical modeling compared to raw character strings?

7. Describe a scenario where you would use the Modulo `%%` operator in data cleaning.

8. Why does the R community strongly prefer `<-` over `=` for variable assignment?

9. What is the difference between single `&` and double `&&` logical operators in R?

10. If you are importing a massive CSV file and a numeric column imports as 'character', what is your debugging process?

IT STUDY HUB

MODULE WRAP-UP

Module Summary

In Module 2, we mapped the foundational architecture of the R language. We began by learning how to properly define variables using the standard `<-` assignment operator, acting as labeled containers in memory. We explored the core atomic data types: Numeric (and Integers), Characters (text strings), and Logicals (booleans). We delved into Factors, learning how R optimizes categorical data for statistical models. Crucially, we analyzed the special states of data—NA, NULL, NaN, Inf—and learned why using `is.na()` is mandatory for detecting missing information. Finally, we mastered the manipulation of these data types using Arithmetic, Relational, and Logical operators, culminating in a hands-on Student Marks Calculator that tied theory directly to practice. You are now equipped to manipulate the raw materials of data science.

IT STUDY HUB

Key Terms Dictionary

Term	Definition
Variable	A named storage location in memory holding a specific value.
Numeric	The default R data type for numbers (doubles, handling decimals).
Logical	Boolean values indicating binary states: TRUE or FALSE.
Factor	A data type optimized for categorical variables with fixed 'levels'.
Coercion	The act of changing a value from one data type to another (e.g., as.numeric).
NA	'Not Available'; explicitly missing data.
Relational Operator	Operators (>, <, ==, !=) that compare two values and output a Logical.
Logical Operator	Operators (&, |, !) that combine multiple TRUE/FALSE conditions.

Quick Revision Checklist

- ✅ Do you know how to assign a variable properly using `<-`?
- ✅ Can you identify the difference between a Character and a Factor?
- ✅ Do you know the difference between NA, NULL, and NaN?
- ✅ Do you understand why `is.na(x)` is required instead of `x == NA`?
- ✅ Can you write an expression that evaluates to TRUE using both `>` and `&`?

Beginner Coding Challenges

Challenge 1: The Detective
Create a variable `mystery <- "100"`. Check its class. Coerce it into a numeric. Check its class again. Multiply it by 5.

Challenge 2: The Bouncer
Create a variable `age <- 17` and `fake_id <- TRUE`. Write a logical expression checking if the person is over 18 OR has a fake ID. Store the result in `can_enter` and print it.

Challenge 3: The Mathematician
Use the Modulo operator (`%%`) to determine if the number 2024 is even or odd. (Hint: divide by 2, what is the remainder?)

WHAT COMES NEXT:

MODULE 3 — DATA STRUCTURES IN R

Now that you understand individual data types, the next module explores how to group them together into Vectors, Lists, Matrices, and the mighty Data Frame.